

Module 07: Communication — Implementing Multi-Agent Inference

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Implement **two communicating agents** whose actions are each other's observations.
2. Code **generalized synchrony** measures between agent beliefs.
3. Simulate a conversation and track belief convergence.

Key Concepts

1. Two-Agent System

```

import numpy as np

class CommunicatingAgent:
    """Agent that can send/receive messages to/from another agent."""

    def __init__(self, A, B, C, D, name="Agent"):
        self.A = A
        self.B = B
        self.C = C
        self.D = D.copy()
        self.qs = D.copy()
        self.name = name
        self.belief_history = []
        self.message_history = []

    def infer_states(self, observation):
        """Update beliefs from observation."""
        self.qs = self.A[observation, :] * self.qs
        self.qs /= self.qs.sum()
        self.belief_history.append(self.qs.copy())
        return self.qs

    def generate_message(self):
        """Generate a message (action) based on current beliefs.

        Message reflects the agent's most likely state belief.
        """
        message = np.argmax(self.qs)
        self.message_history.append(message)
        return message

```

```
def receive_message(self, message):  
    """Process a message from another agent as an observation."""  
    return self.infer_states(message)
```

2. Communication Simulation

```

def simulate_conversation(agent_a, agent_b, shared_state, num_rounds=10):
    """
    Two agents exchange messages about a shared hidden state.

    Parameters:
        agent_a, agent_b: CommunicatingAgent instances
        shared_state: the true state they're communicating about
        num_rounds: number of message exchanges
    """
    # Initial private observations
    obs_a = np.random.choice(agent_a.A.shape[0], p=agent_a.A[:, shared_st
    obs_b = np.random.choice(agent_b.A.shape[0], p=agent_b.A[:, shared_st

    agent_a.infer_states(obs_a)
    agent_b.infer_states(obs_b)

    print(f"Initial: A={agent_a.qs.round(3)}, B={agent_b.qs.round(3)}")

    for round in range(num_rounds):
        # A sends message to B
        msg_a = agent_a.generate_message()
        agent_b.receive_message(msg_a)

        # B sends message to A
        msg_b = agent_b.generate_message()
        agent_a.receive_message(msg_b)

        sync = compute_synchrony(agent_a.qs, agent_b.qs)
        print(f"Round {round}: A={agent_a.qs.round(3)}, "
              f"B={agent_b.qs.round(3)}, Sync={sync:.3f}")

```

```
return agent_a, agent_b
```

3. Measuring Generalized Synchrony

```
def compute_synchrony(beliefs_a, beliefs_b):
    """
    Compute synchrony as 1 - Jensen-Shannon divergence.

    Returns value in [0, 1] where 1 = perfect agreement.
    """
    m = 0.5 * (beliefs_a + beliefs_b)

    kl_a = (beliefs_a * (np.log(beliefs_a + 1e-16) - np.log(m + 1e-16)))
    kl_b = (beliefs_b * (np.log(beliefs_b + 1e-16) - np.log(m + 1e-16)))

    jsd = 0.5 * kl_a + 0.5 * kl_b # JSD in [0, log(2)]
    synchrony = 1.0 - jsd / np.log(2) # normalize to [0, 1]

    return synchrony

def track_synchrony(agent_a, agent_b):
    """Track synchrony over the conversation."""
    T = min(len(agent_a.belief_history), len(agent_b.belief_history))
    syncs = []
    for t in range(T):
        s = compute_synchrony(agent_a.belief_history[t], agent_b.belief_h
        syncs.append(s)
    return syncs
```

4. Shared Priors (Cultural Norms)

```
def create_agents_with_shared_priors(shared_D, A_a, A_b, B, C):  
    """Create agents with identical priors (cultural norms)."""  
    agent_a = CommunicatingAgent(A_a, B, C, shared_D, name="A")  
    agent_b = CommunicatingAgent(A_b, B, C, shared_D, name="B")  
    return agent_a, agent_b  
  
def create_agents_with_different_priors(D_a, D_b, A_a, A_b, B, C):  
    """Create agents with different priors (cultural mismatch)."""  
    agent_a = CommunicatingAgent(A_a, B, C, D_a, name="A")  
    agent_b = CommunicatingAgent(A_b, B, C, D_b, name="B")  
    return agent_a, agent_b
```

Summary

Multi-agent Active Inference implements communication as coupled inference — agents exchange messages that serve as observations for each other. Generalized synchrony is measured using Jensen-Shannon divergence. Shared priors (cultural norms) facilitate faster convergence.

Further Reading

- Friston, K. J. & Frith, C. D. (2015). A duet for one. *Consciousness and Cognition*, 36, 390-405.
- Vasil, J. et al. (2020). A world unto itself: Human communication as active inference. *Frontiers in Psychology*, 11, 417.